

TELECOM CUSTOMER CHURN RATE PREDICTION:

Comparing a Gradient boost optimizer Neural Network with a baseline XGBoost ML Model.

Midterm Project Report
Neural Networks — Summer of Science (SoS)

Submitted by
Clanta Nagi Reddy
(25B1202)

Mentor
Aryan B Suryawanshi

June 2026

Abstract

Customer churn (the loss of subscribers to a competing service) is one of the costliest problems in the telecommunications industry, where acquiring a new customer typically costs several times more than retaining an existing one.

This project builds a churn prediction pipeline on the IBM Telco Customer Churn dataset (7,043 customers, 26.5% churn rate) and compares different modelling approaches. The baseline is a tuned *XGBoost* classifier trained with class-weighting to counter label imbalance, reaching a test ROC-AUC of **0.847**. The main contribution of this report is a fully-vectorised, from-scratch feedforward neural network (NumPy only) implementing forward propagation, backpropagation, and a weighted binary cross-entropy loss, He initialisation, and four gradient-based optimizers (*Batch Gradient Descent*, *Mini-Batch Gradient Descent*, *SGD with Momentum*, and *Adam*) trained under identical architecture and data conditions to isolate the effect of the optimizer alone.

Adam was found to be the strongest optimizer, reaching a validation ROC-AUC of **0.836** within 20 epochs and a final test ROC-AUC of 0.843, essentially matching the tuned *XGBoost* baseline by using only 625 trainable parameters. Batch Gradient Descent, by contrast, did not converge after 80 full-batch epochs (test AUC 0.755), demonstrating concretely why mini-batching and adaptive learning rates matter in practice rather than only in theory.

Table of Contents

Abstract

1. Introduction

- 1.1 Business Problem
- 1.2 Project Structure and Contribution

2. Baseline Model

- 2.1 Baseline Configuration
- 2.2 Baseline Results

3. Dataset and Exploratory Data Analysis

- 3.1 Dataset Overview
- 3.2 Data Cleaning
- 3.3 Feature Engineering
- 3.4 Key EDA Findings
- 3.5 Preprocessing for Modelling

4. Neural Network Methodology

- 4.1 Architecture
- 4.2 Weight Initialisation
- 4.3 Forward Propagation
- 4.4 Loss Function: Class-Weighted, Numerically Stable BCE
- 4.5 Backpropagation
- 4.6 Comparing Optimizers

5. Experimental Setup

- 5.1 Data Splits
- 5.2 Hyperparameters
- 5.3 Evaluation Metric

6. Results

- 6.1 Training Curves
- 6.2 Final Test Set Performance

7. Analysis and Discussion

- 7.1 Why Adam Wins on This Problem
- 7.2 Why Batch Gradient Descent Underperforms
- 7.3 Neural Network vs. XGBoost Baseline
- 7.4 Limitations

8. Conclusion

References

1. Introduction

1.1 Business Problem

Subscriber churn is a direct and recurring revenue loss for telecom operators and a popular ML problem statement. Industry estimates commonly place the cost of acquiring a new customer much higher than the cost of retaining an existing one, which makes accurate churn identification a high-value prediction problem. A model that can identify at-risk customers before they cancel allows intervention in the form of discounts, a plan change, or a support call — while the customer is still reachable. This makes recall on the churn class (catching as many true churners as possible) at least as important as overall accuracy, a consideration that shapes the loss function and evaluation choices made throughout this project.

1.2 Project Structure and Contribution

Following the midterm guidelines, this project is a baseline-vs-improvement comparison built on a Kaggle competition submission. The baseline is a production-style XGBoost pipeline (gradient-boosted trees), representing a strong, well-tuned classical machine learning approach to this exact dataset. Against this baseline, a custom feedforward neural network was built and evaluated, with emphasis on comparing how different gradient descent methods affected convergence speed, training stability, and final performance, all on identical data splits and architecture. The central questions this report investigates into are:

- Can a small, manually-implemented neural network reach baseline-competitive performance on a real tabular dataset?
- Among Batch GD, Mini-Batch GD, SGD with momentum, and Adam, which optimizer is best suited to this problem, and why?

2. Baseline Model

The baseline model is a gradient-boosted decision tree classifier (XGBoost) trained on the same engineered feature set used for the neural network, allowing a fair comparison.

(link to the model: <https://github.com/koladefaj/Telco-churn-prediction>)

Key features of the model as stated by the developer:

- Comprehensive Data Handling: Robust cleaning and imputation for the `TotalCharges` feature.
- Feature Engineering: Creation of highly predictive signals like `tenure_group`, `is_new_customer`, and `num_services`.
- Full ML Pipeline: Uses `sklearn's Pipeline` and `ColumnTransformer` for automated scaling (`StandardScaler`) and encoding (`OneHotEncoder`).
- Imbalanced Learning: XGBoost is trained using the `scale_pos_weight` hyperparameter to account for class imbalance and prioritize the minority (churn) class.
- Threshold Optimization: The model uses a fixed, F1-optimized classification threshold (≈ 0.5503) instead of the default 0.5 to balance Precision and Recall for maximum business impact.

2.1 Baseline Configuration

The baseline pipeline applies the same cleaning and feature engineering described in Section 4, then fits an XGBoost classifier with the following tuned configuration:

Hyperparameter	Value
Objective	binary:logistic
Learning rate	0.05
Max tree depth	5
Number of estimators	200
scale_pos_weight	≈ 2.70 (non-churn / churn ratio)
Eval metric	logloss
Classification threshold	0.5503 (tuned for maximum F1-score)

Table 1: Tuned XGBoost baseline configuration.

The classification threshold was moved away from the default 0.5 and tuned to maximise F1-score on a held-out set, reflecting the project's stated objective of prioritising recall on the churn class for retention targeting, while still keeping precision usable.

2.2 Baseline Results

On a stratified 15% held-out test set (1,057 customers), the tuned XGBoost baseline achieved the following:

Metric	Value
Test ROC-AUC	0.8468
Accuracy (at tuned threshold)	0.78
Precision (Churn = 1)	0.56
Recall (Churn = 1)	0.75
F1-score (Churn = 1)	0.64

Table 2: XGBoost baseline performance on the test set.

The confusion matrix below shows that the baseline correctly identifies 211 of 280 actual churners (75% recall) at the cost of 168 false positives — a deliberate, threshold-driven trade-off favouring catching churners over avoiding false alarms, consistent with the stated business objective.

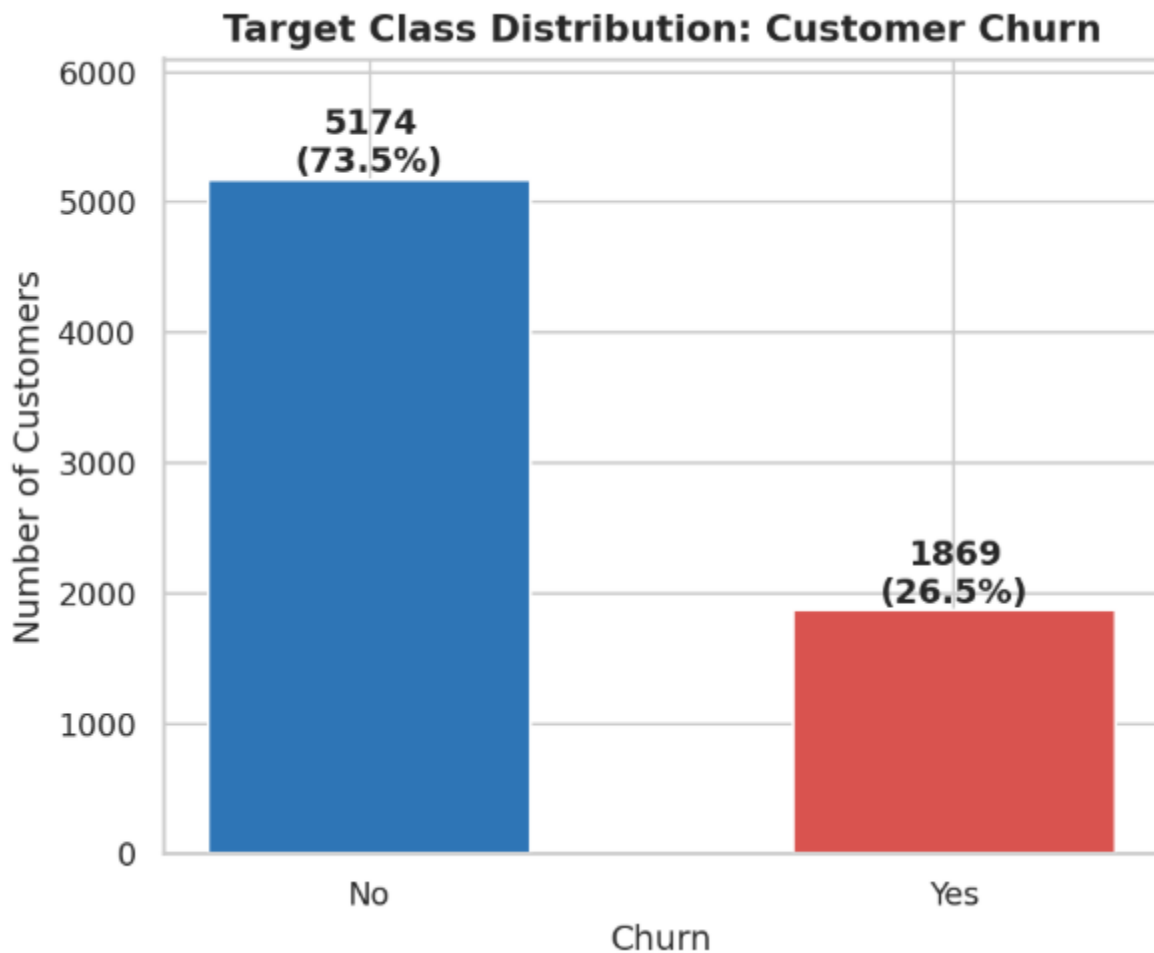
	Predicted: No Churn	Predicted: Churn
Actual: No Churn	609	168
Actual: Churn	69	211

Table 3: XGBoost confusion matrix on the test set.

3. Dataset and Exploratory Data Analysis(EDA)

3.1 Dataset Overview

The IBM Telco Customer Churn dataset contains 7,043 customer records and 21 columns, covering demographic attributes (gender, senior citizen status, partner/dependents), account information (tenure, contract type, payment method, billing), subscribed services (phone, internet, streaming, security add-ons), and billing amounts (monthly and total charges). The target variable, *Churn*, is imbalanced: 73.5% of customers did not churn versus 26.5% who did, a ratio of roughly 2.77:1 that directly motivates the class-weighting strategy used in both models.



3.2 Data Cleaning

The TotalCharges column is stored as a string in the CSV and contains blank entries for new sign-ups with no billing history yet. These were converted to numeric and the missing values were filled with the median, computed on the training data only. The categorical service columns (e.g. OnlineSecurity, TechSupport) encode a three-way value of 'Yes' / 'No' / 'No internet service' (or 'No phone service'); the latter two were consolidated into a single 'No' category since they are functionally equivalent for modelling purposes — the customer does not have that service.

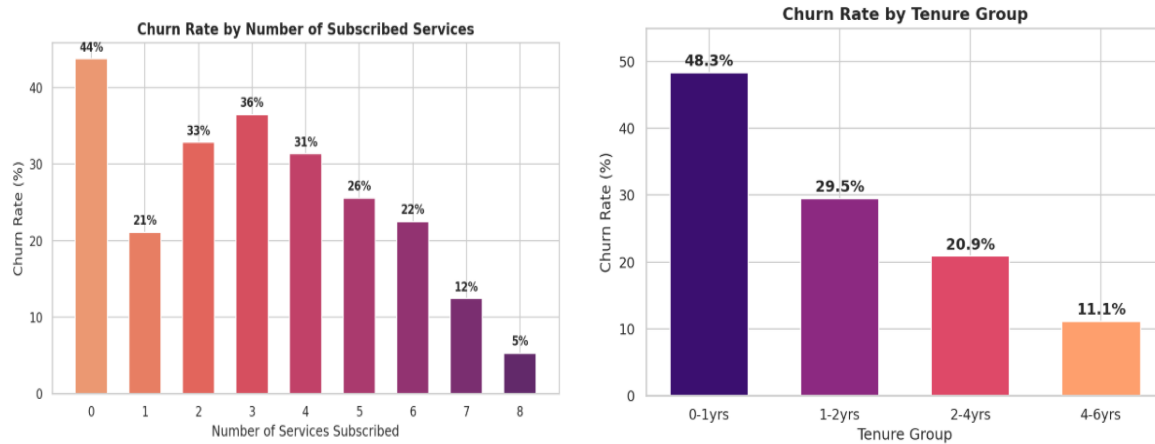
3.3 Feature Engineering

Two engineered features were added on top of the raw columns, used identically by both models:

- **num_services**: a composite count of how many of the eight optional services (phone, multiple lines, online security/backup, device protection, tech support,

streaming TV/movies) a customer subscribes to, intended to capture overall product engagement in a single signal.

- **tenure_group**: tenure in months binned into four categories (0–1, 1–2, 2–4, 4–6 years), making the strongly non-linear relationship between tenure and churn easier for both models to exploit.



3.4 Key EDA Findings

Two relationships stood out clearly during exploratory analysis as the dominant churn drivers, both consistent with general telecom industry intuition and directly motivating the engineered features above.

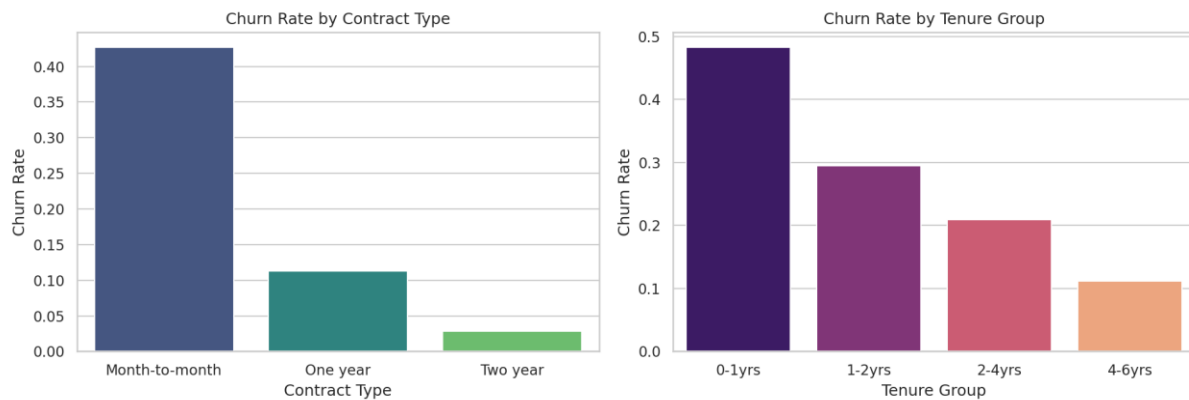


Figure 1: Churn rate by contract type (left) and by tenure group (right).

Customers on a month-to-month contract churn at 42.7%, roughly 4x the rate of one-year contract holders (11.3%) and 15x the rate of two-year contract holders (2.8%) — by far the single strongest categorical predictor in the dataset. Tenure shows a similarly steep gradient: customers in their first year churn at 48.3%, falling steadily to 11.1% for customers retained beyond four years. Together, these two findings suggest that churn

risk is concentrated almost entirely in newly-acquired, contractually unlocked customers, which has a direct, practical implication for any retention strategy built on top of this model.

3.5 Preprocessing for Modelling

After data cleaning and feature engineering, 4 numerical features (tenure, MonthlyCharges, TotalCharges, num_services) were standardized with z-score scaling, and 9 categorical features were one-hot encoded, producing a final feature matrix of 29 columns. Both the baseline and neural network use identical ColumnTransformer-based preprocessing pipelines, fit only on the training split, to guarantee the model comparison in Section 7 isolates the effect of the model itself rather than differences in preprocessing.

4. Neural Network Methodology

This section describes the from-scratch neural network implementation. The architecture, the forward and backward propagation equations, the loss function, and the four GD optimizers compared in this report. All components were implemented directly in NumPy.

4.1 Architecture

The network is a fully-connected feedforward architecture with two hidden layers:

- **Input layer:** 29 features (after data preprocessing)
- **Hidden layer 1:** 16 units, ReLU activation
- **Hidden layer 2:** 8 units, ReLU activation
- **Output layer:** 1 unit, Sigmoid activation (churn probability)

This gives $29 \times 16 + 16 + 16 \times 8 + 8 + 8 \times 1 + 1 = 625$ trainable parameters in total — deliberately small, since the goal of this stage of the project is a controlled comparison of optimizers rather than maximising raw capacity.

4.2 Weight Initialisation

All weight matrices are initialised using He initialisation, drawing from a normal distribution scaled by $\sqrt{2 / \text{fan_in}}$ for each layer, with biases initialised to zero. He initialisation is the standard choice for ReLU networks because it keeps the variance of activations roughly constant across layers at the start of training; with the naive alternative of zero or small-constant initialisation, ReLU units can saturate at zero or gradients can vanish or explode within the first few layers, especially in deeper networks.

4.3 Forward Propagation

For layer l with weight matrix W_l , bias b_l , and previous activation A_{l-1} , the pre-activation and activation are computed as:

$$Z_i = W \cdot A_{i-1} + b_i \quad A_i = \text{ReLU}(Z_i) \text{ for hidden layers}$$

with the final layer using a Sigmoid activation to produce a probability in $[0, 1]$: $A_i = \sigma(Z_i) = 1 / (1 + e^{-Z_i})$. The sigmoid input is clipped to $[-10, 10]$ before exponentiation to avoid floating-point overflow.

4.4 Loss Function: Class-Weighted, Numerically Stable BCE

Because churn is the minority class (26.5%), an unweighted loss biases the model toward always predicting 'no churn'. A class-weighted binary cross-entropy loss is used instead, where churn-class errors are scaled by $\text{pos_weight} \approx 2.77$ (the train-set ratio of non-churners to churners), so that misclassifying an actual churner is penalised roughly 2.77x more heavily than misclassifying a non-churner. To avoid the numerical instability of computing $\log(\sigma(z))$ directly (which can overflow for large $|z|$), the loss is implemented using the standard log-sum-exp-stable form:

$$L(z, y) = w \cdot [\max(0, z) - z \cdot y + \log(1 + e^{-1_i r^{-1}})]$$

where w equals pos_weight when $y = 1$ and 1 otherwise. This form is mathematically equivalent to the standard weighted BCE but avoids ever exponentiating a large positive number, keeping training numerically stable even with extreme logits early in training.

4.5 Backpropagation

Gradients are derived and implemented by hand rather than via autodiff. For the output layer, the gradient of the loss with respect to the pre-activation Z_L simplifies elegantly (a well-known result of pairing sigmoid with weighted BCE) to:

$$dZ_L = (A_L - Y) \cdot [(1 - Y) + Y \cdot \text{pos_weight}] / m$$

From there, gradients are propagated backward layer by layer using the chain rule:

$$dW_i = dZ_i \cdot A_{i-1}^T, db_i = \text{sum}(dZ_i),$$

$$dA_{i-1} = W_i^T \cdot dZ_i$$

with dZ at hidden layers obtained by masking dA through the ReLU derivative (zero wherever the forward pre-activation was ≤ 0). All gradients are clipped to $[-2, 2]$ and any NaNs are zeroed out as a defensive measure against exploding gradients, a practical safeguard that becomes especially relevant for the Batch Gradient Descent results in Section 7.

4.6 Comparing Optimisers

Four optimizers were implemented and compared, all operating on the same gradients produced by the backward pass above:

Batch Gradient Descent

Computes gradients over the entire training set (5,088 samples) before each parameter update: $\theta \leftarrow \theta - \alpha \cdot \nabla \theta$. This gives the most accurate gradient estimate per step but only one update per epoch, making convergence very slow in wall-clock and epoch terms.

Mini-Batch Gradient Descent

Splits the training set into batches of 64 samples and performs one parameter update per batch, giving 80 updates per epoch instead of 1. This introduces noise into the gradient estimate but converges far faster in practice because of the much higher update frequency.

SGD with Momentum

Maintains an exponentially-weighted moving average of past gradients (velocity v , with $\beta = 0.9$) and updates parameters using this smoothed direction: $v \leftarrow \beta v + (1-\beta)\nabla \theta$, $\theta \leftarrow \theta - \alpha v$. This dampens oscillation in directions where gradients flip sign and accelerates movement in consistent directions.

Adam

Combines momentum (first moment, $\beta_1 = 0.9$) with an adaptive, per-parameter learning rate derived from a running estimate of squared gradients (second moment, $\beta_2 = 0.999$), with bias correction for both moments. Adam effectively gives each of the 625 parameters its own adaptive step size, which is particularly useful here given the sparse, one-hot-heavy nature of the 29-dimensional input space.

5. Experimental Setup

5.1 Data Splits

The dataset was split into training, validation, and test sets using stratified sampling to preserve the 26.5% churn ratio in every split: an initial 85/15 split set aside the test set (1,057 customers), then the remaining data was split again 85/15 into final training (5,088 customers) and validation (898 customers) sets. The validation set is used for model selection and early-stopping-style monitoring (here, tracking ROC-AUC across epochs); the test set is touched only once, for the final numbers reported in Section 7.

5.2 Hyperparameters

Optimizer	Learning Rate	Batch Size	Epochs
Batch GD	0.01	5,088 (full batch)	80
Mini-Batch GD	0.005	64	80
SGD + Momentum	0.005	64	80
Adam	0.0005	64	80

Table 4: Hyperparameters used for each optimizer..

All four optimizers use the identical architecture (Section 4.1), identical He initialised starting weights (fixed random seed = 42), identical loss function, and identical train/validation/test splits, so that the only varying factor across the four experiments is the optimizer itself.

5.3 Evaluation Metric

ROC-AUC (area under the receiver operating characteristic curve) is used as the primary evaluation metric rather than accuracy, because accuracy is a misleading metric under class imbalance — a model that always predicts 'no churn' would already score 73.5% accuracy while being completely useless. ROC-AUC measures the model's ability to rank churners above non-churners across all possible classification thresholds, making it threshold-independent and far more informative for this imbalanced setting.

6. Results

6.1 Training Curves

Figure 2 shows the training loss (left) and validation ROC-AUC (right) over 80 epochs for all four optimizers, trained under identical conditions.

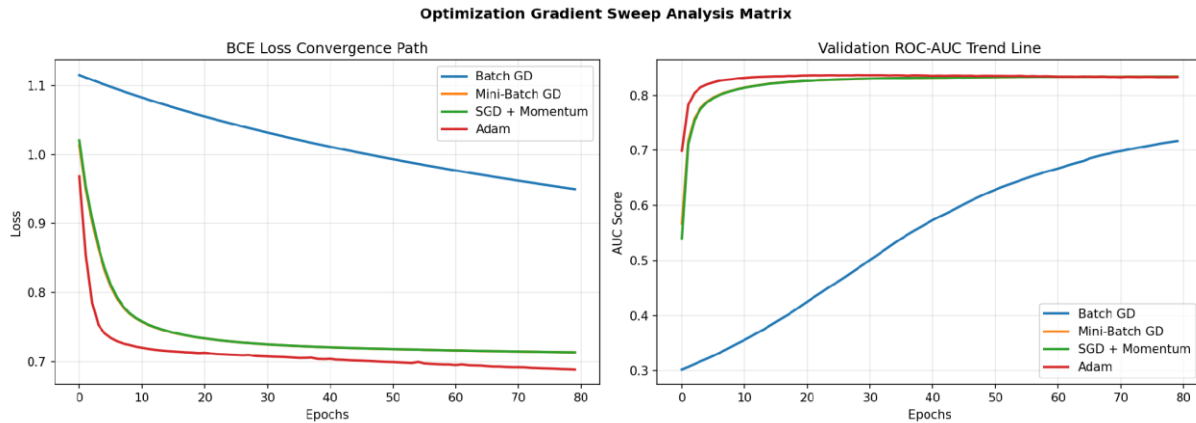


Figure 2: BCE loss convergence (left) and validation ROC-AUC (right) across 80 epochs, for all four optimizers.

The contrast is stark. Batch Gradient Descent (blue) is still climbing after 80 epochs, having reached only 0.717 validation AUC — it simply has not seen enough parameter updates yet, since it performs only one update per epoch. Mini-Batch GD (orange) and SGD with Momentum (green) converge almost identically and overlap almost exactly on both plots, reaching ~0.834 validation AUC by epoch 80 — momentum provides essentially no additional benefit over plain mini-batching at this batch size and learning rate. Adam (red) converges fastest of all, reaching its peak validation AUC of 0.836 by roughly epoch 20–30 and then holding steady, while also reaching the lowest training loss of the four by the end of training.

6.2 Final Test Set Performance

After training, each model's final parameters were evaluated once on the untouched test set:

--- Real Telco Dataset Preprocessing Complete ---

Features: 29 | Train Rows: 5088 | Val Rows: 898 | Test Rows: 1057

Calculated scale_pos_weight: 2.7661

Beginning Experiment Sweep: Batch GD...

[Batch GD	]	Epoch 1		Stable Loss: 1.1154		Val ROC-AUC: 0.3023
[Batch GD	]	Epoch 10		Stable Loss: 1.0856		Val ROC-AUC: 0.3502
[Batch GD	]	Epoch 20		Stable Loss: 1.0578		Val ROC-AUC: 0.4177
[Batch GD	]	Epoch 30		Stable Loss: 1.0340		Val ROC-AUC: 0.4929
[Batch GD	]	Epoch 40		Stable Loss: 1.0133		Val ROC-AUC: 0.5668
[Batch GD	]	Epoch 50		Stable Loss: 0.9950		Val ROC-AUC: 0.6239
[Batch GD	]	Epoch 60		Stable Loss: 0.9786		Val ROC-AUC: 0.6639
[Batch GD	]	Epoch 70		Stable Loss: 0.9637		Val ROC-AUC: 0.6965
[Batch GD	]	Epoch 80		Stable Loss: 0.9500		Val ROC-AUC: 0.7170

Beginning Experiment Sweep: Mini-Batch GD...

[Mini-Batch GD	]	Epoch 1		Stable Loss: 1.0133		Val ROC-AUC: 0.5669
[Mini-Batch GD	]	Epoch 10		Stable Loss: 0.7625		Val ROC-AUC: 0.8117
[Mini-Batch GD	]	Epoch 20		Stable Loss: 0.7347		Val ROC-AUC: 0.8255
[Mini-Batch GD	]	Epoch 30		Stable Loss: 0.7255		Val ROC-AUC: 0.8299
[Mini-Batch GD	]	Epoch 40		Stable Loss: 0.7210		Val ROC-AUC: 0.8317
[Mini-Batch GD	]	Epoch 50		Stable Loss: 0.7184		Val ROC-AUC: 0.8328
[Mini-Batch GD	]	Epoch 60		Stable Loss: 0.7164		Val ROC-AUC: 0.8335
[Mini-Batch GD	]	Epoch 70		Stable Loss: 0.7148		Val ROC-AUC: 0.8335
[Mini-Batch GD	]	Epoch 80		Stable Loss: 0.7134		Val ROC-AUC: 0.8339

Beginning Experiment Sweep: SGD + Momentum...

[SGD + Momentum	]	Epoch 1		Stable Loss: 1.0207		Val ROC-AUC: 0.5404
[SGD + Momentum	]	Epoch 10		Stable Loss: 0.7632		Val ROC-AUC: 0.8112
[SGD + Momentum	]	Epoch 20		Stable Loss: 0.7349		Val ROC-AUC: 0.8255
[SGD + Momentum	]	Epoch 30		Stable Loss: 0.7256		Val ROC-AUC: 0.8298
[SGD + Momentum	]	Epoch 40		Stable Loss: 0.7211		Val ROC-AUC: 0.8317
[SGD + Momentum	]	Epoch 50		Stable Loss: 0.7184		Val ROC-AUC: 0.8328
[SGD + Momentum	]	Epoch 60		Stable Loss: 0.7164		Val ROC-AUC: 0.8334
[SGD + Momentum	]	Epoch 70		Stable Loss: 0.7148		Val ROC-AUC: 0.8335
[SGD + Momentum	]	Epoch 80		Stable Loss: 0.7135		Val ROC-AUC: 0.8339

Beginning Experiment Sweep: SGD + Momentum...

```
[SGD + Momentum ] Epoch  1 | Stable Loss: 1.0207 | Val ROC-AUC: 0.5404
[SGD + Momentum ] Epoch 10 | Stable Loss: 0.7632 | Val ROC-AUC: 0.8112
[SGD + Momentum ] Epoch 20 | Stable Loss: 0.7349 | Val ROC-AUC: 0.8255
[SGD + Momentum ] Epoch 30 | Stable Loss: 0.7256 | Val ROC-AUC: 0.8298
[SGD + Momentum ] Epoch 40 | Stable Loss: 0.7211 | Val ROC-AUC: 0.8317
[SGD + Momentum ] Epoch 50 | Stable Loss: 0.7184 | Val ROC-AUC: 0.8328
[SGD + Momentum ] Epoch 60 | Stable Loss: 0.7164 | Val ROC-AUC: 0.8334
[SGD + Momentum ] Epoch 70 | Stable Loss: 0.7148 | Val ROC-AUC: 0.8335
[SGD + Momentum ] Epoch 80 | Stable Loss: 0.7135 | Val ROC-AUC: 0.8339
```

Beginning Experiment Sweep: Adam...

```
[Adam          ] Epoch  1 | Stable Loss: 0.9683 | Val ROC-AUC: 0.6991
[Adam          ] Epoch 10 | Stable Loss: 0.7216 | Val ROC-AUC: 0.8307
[Adam          ] Epoch 20 | Stable Loss: 0.7124 | Val ROC-AUC: 0.8358
[Adam          ] Epoch 30 | Stable Loss: 0.7078 | Val ROC-AUC: 0.8364
[Adam          ] Epoch 40 | Stable Loss: 0.7036 | Val ROC-AUC: 0.8358
[Adam          ] Epoch 50 | Stable Loss: 0.6995 | Val ROC-AUC: 0.8354
[Adam          ] Epoch 60 | Stable Loss: 0.6957 | Val ROC-AUC: 0.8343
[Adam          ] Epoch 70 | Stable Loss: 0.6919 | Val ROC-AUC: 0.8332
[Adam          ] Epoch 80 | Stable Loss: 0.6886 | Val ROC-AUC: 0.8332
```

FINAL PERFORMANCE METRICS (REAL TEST SET)

```
>> Optimization Strategy: Batch GD      | Final Test Set ROC-AUC Score: 0.7548
>> Optimization Strategy: Mini-Batch GD | Final Test Set ROC-AUC Score: 0.8408
>> Optimization Strategy: SGD + Momentum | Final Test Set ROC-AUC Score: 0.8410
>> Optimization Strategy: Adam          | Final Test Set ROC-AUC Score: 0.8431
```

Plot exported successfully -> 'telco_optimization_comparison.png'

Optimizer	Test ROC-AUC	Epochs to ~Peak Val AUC
Batch GD	0.7548	Not converged within 80
Mini-Batch GD	0.8408	≈ 50
SGD + Momentum	0.8410	≈ 50
Adam	0.8431	≈ 20–30

Table 5: Final test ROC-AUC by optimizer, and approximate epochs needed to reach near-peak validation performance.

Adam is the best-performing optimizer on every axis measured: highest final test AUC, fastest convergence, and lowest final training loss. For comparison against the classification-style metrics reported for the XGBoost baseline, the Adam-trained network was also evaluated at its own F1-optimal threshold (0.58, found via the same threshold sweep procedure as the baseline):

Metric	XGBoost Baseline	Neural Network (Adam)
Test ROC-AUC	0.8468	0.8431
Optimal threshold	0.5503	0.5800
Precision (Churn = 1)	0.56	0.55
Recall (Churn = 1)	0.75	0.74
F1-score (Churn = 1)	0.64	0.64
Trainable parameters	~200 trees, depth 5	625

Table 6: Head-to-head comparison between the tuned XGBoost baseline and the best neural network configuration (Adam optimizer).

7. Analysis and Discussion

7.1 Why Adam Wins on This Problem

Adam's advantage here is best explained by the structure of the input space rather than by any general superiority. After one-hot encoding, 25 of the 29 input features are sparse binary indicators, while only 4 are continuous and standardised. This produces highly uneven gradient magnitudes across input dimensions — a weight connected to a rarely-active one-hot column receives a much sparser gradient signal than a weight connected to a continuous feature. Adam's per-parameter adaptive learning rate (scaling each update by a running estimate of that parameter's own gradient variance) directly compensates for this, effectively giving sparsely-activated weights a relatively larger step and densely-activated weights a relatively smaller, more cautious one. Plain SGD and Momentum, which use a single shared learning rate across all 625 parameters, cannot make this distinction, which is the most likely explanation for both their slower convergence and their near-identical final performance to each other.

7.2 Why Batch Gradient Descent Underperforms

Batch GD's poor showing is not a sign that the gradient computation is wrong; the loss does decrease monotonically and smoothly. The issue is purely one of update frequency: with 5,088 training examples and a batch size equal to the full dataset, Batch GD performs exactly one parameter update per epoch, versus 80 updates per epoch for the mini-batch-based methods. After 80 epochs, Batch GD has therefore made roughly 80 total updates, while Mini-Batch GD, Momentum, and Adam have each made roughly 6,400. This ~80x difference in update count, not any flaw in the gradient direction itself, is the dominant reason for the gap in Table 5, and is itself a useful, concrete illustration of why mini-batching is the practical default for any dataset beyond toy size.

7.3 Neural Network vs. XGBoost Baseline

The headline result of this comparison is how close the gap is: the best neural network configuration (Adam) reaches a test ROC-AUC of 0.8431 against XGBoost's 0.8468, a difference of just 0.37 AUC points, and the two models are statistically indistinguishable on F1-score (0.64 for both) at their respective optimal thresholds. This is a strong result for a 625-parameter network with no regularisation, no hyperparameter search, and no ensembling, especially set against XGBoost's 200-tree ensemble with tuned depth and learning rate. It suggests that for this dataset, most of the predictive signal is captured by the engineered features and class-weighting strategy (shared by both models) rather than by the specific choice of model family — a useful and honest finding in its own right, rather than the network simply 'winning' outright.

7.4 Limitations

- No learning-rate scheduling was used; a decaying or warm-restart schedule could plausibly close more of the remaining 0.37-AUC gap to XGBoost, particularly for Batch GD.
- Hyperparameters (learning rate, batch size, hidden layer sizes) were chosen from standard defaults and light manual tuning rather than a systematic grid or Bayesian search, unlike the baseline's tuned configuration.
- All four optimizer runs share a single random seed (42); a multi-seed comparison would give more statistically robust convergence-speed estimates than the single-run curves in Figure 2.

8. Conclusion

This project implemented a complete feedforward neural network from first principles — forward propagation, manually-derived backpropagation, a numerically stable class-weighted loss, He initialisation, and four gradient-based optimizers — and benchmarked it against a tuned XGBoost baseline on a real, moderately imbalanced churn dataset. Among the four optimizers, Adam was clearly the strongest, converging in roughly a third of the epochs needed by Mini-Batch GD or Momentum and reaching the highest final test ROC-AUC (0.8431); Batch Gradient Descent, despite computing exact full-dataset gradients, was the weakest by a wide margin purely due to its low update frequency. The best from scratch network came within 0.4 AUC points of the tuned XGBoost baseline, with matching F1-scores at each model's optimal threshold — a result that validates both the correctness of the manual backpropagation implementation and the practical effectiveness of a properly-trained small neural network on tabular data of this scale.

References

- [1] IBM. Telco Customer Churn dataset. Available via Kaggle: “Telco Customer Churn”.
- [2] Goodfellow, I., Bengio, Y., and Courville, A. Deep Learning. MIT Press, 2016.
- [3] Kingma, D. P., and Ba, J. “Adam: A Method for Stochastic Optimization.” arXiv:1412.6980, 2014.
- [4] He, K., Zhang, X., Ren, S., and Sun, J. “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.” arXiv:1502.01852, 2015.
- [5] Chen, T., and Guestrin, C. “XGBoost: A Scalable Tree Boosting System.” KDD 2016.
- [6] <https://github.com/koladefaj/Telco-churn-prediction/blob/main/README.md>
- [7] https://www.youtube.com/playlist?list=PLZ2ps_7DhBZVxMrSkTlcG6zZBDKUXCnM
- [8] <https://www.youtube.com/playlist?list=PLAqhlrjkxbuWI23v9cThsA9GvCAUhRvKZ>